Memory Management

Physical address space is the entire range of unique memory addresses that a processor can physically access.

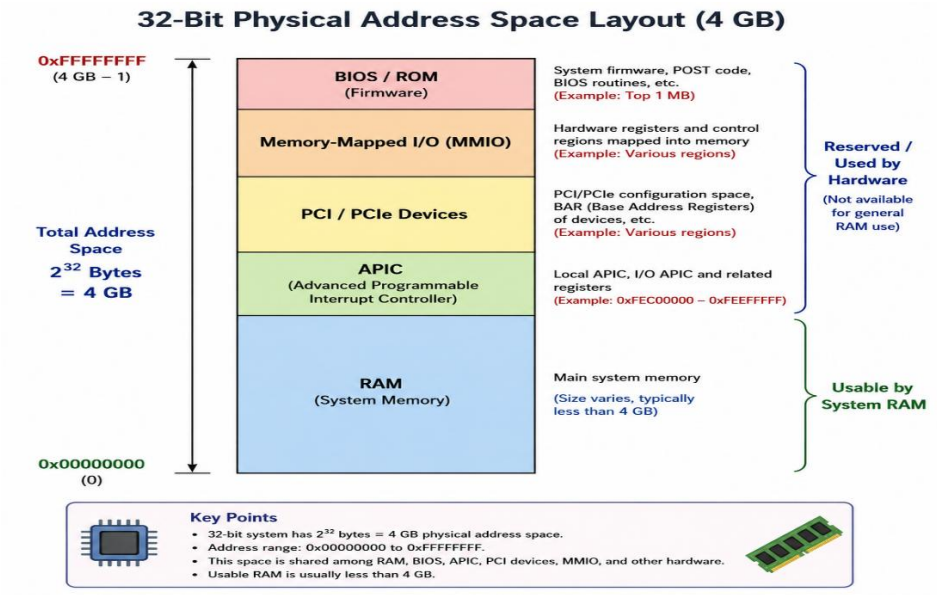
32-Bit System Capacity: A 32-bit system has an address space of 2^{32} bytes, which equals exactly **4 GB** of addressable space.

Maximum Address Boundary: The address range for a 32-bit system spans from 0x00000000 up to the top limit of 0xFFFFFFFF.

Shared Allocation: This 4 GB space is not reserved exclusively for system RAM; it must be shared among multiple hardware components.

System Users: The physical address space is actively utilized by:

- **RAM:** The main system memory.
- **BIOS:** The system's basic input/output firmware.
- **APIC:** The Advanced Programmable Interrupt Controller.
- **PCI:** Peripheral Component Interconnect slots and buses.
- **Memory-Mapped I/O (MMIO):** Other hardware peripherals that map their control registers directly into the processor's memory map for communication.



-

Sudo cat /proc/iomem : it will give memory map of Physical System

```
sethuraj@qbs-generic-lin-2204:/$ cd /proc/
sethuraj@qbs-generic-lin-2204:/proc$ ls
1      10453  11673  12137  185  281  41   491  516  6    67    855  965        devices       kallsyms      misc         stat             zoneinfo
10     10708  11754  12150  186  29   416  492  52   60   68    864  972        diskstats     kcore         modules      swaps
1010   10711  11770  12161  19   3    419  494  525  61   681   867  974        dma           keys          mounts       sys
1016   10716  11781  12167  2    30   42   5    528  613  682   870  990        driver        key-users     mtrr         sysrq-trigger
1017   1103   11786  12168  20   32   422  50   53   615  687   897  acpi       dynamic_debug kmsg          net          sysvipc
1019   1123   11792  12176  21   329  43   500  538  616  7     908  bootconfig execdomains   kpagecgroup   pagetypeinfo thread-self
1021   115    11830  12183  22   33   44   501  54   62   72    910  buddyinfo  fb            kpagecount    partitions   timer_list
1023   11659  11853  13     225  34   45   504  55   63   73    914  bus        filesystems   kpageflags    pressure     tty
1024   11662  119    14     23   35   46   506  56   64   7705  935  cgroups    fs            latency_stats schedstat    uptime
1026   11663  11916  15     24   36   47   507  57   645  81    940  cmdline    interrupts    loadavg       scsi         version
1034   11670  12     16     26   37   486  51   577  648  825   953  consoles   iomem         locks         self         version_signature
10359  11671  12121  17     27   39   487  511  58   65   83    959  cpuinfo    ioports       mdstat        slabinfo     vmallocinfo
1037   11672  12129  18     28   4    490  512  59   66   854   96   crypto     irq           meminfo       softirqs     vmstat
sethuraj@qbs-generic-lin-2204:/proc$
```

```
sethuraj@qbs-generic-lin-2204:/$ sudo cat /proc/iomem
[sudo] password for sethuraj:
00000000-00000fff : Reserved
00001000-0009fbff : System RAM
0009fc00-0009ffff : Reserved
000a0000-000bffff : PCI Bus 0000:00
000c0000-000c9bff : Video ROM
000ca000-000cadff : Adapter ROM
000cb000-000cd3ff : Adapter ROM
000f0000-000fffff : Reserved
  000f0000-000fffff : System ROM
00100000-ce3d9fff : System RAM
  97000000-985fffff : Kernel code
  98600000-993aafff : Kernel rodata
  99400000-99854ebf : Kernel data
  99d51000-9a1fffff : Kernel bss
ce3da000-ce3fffff : Reserved
ce400000-febfffff : PCI Bus 0000:00
  fc000000-fcffffff : 0000:00:02.0
    fc000000-fcffffff : bochs-drm
  fd000000-fd1fffff : PCI Bus 0000:03
  fd200000-fd3fffff : PCI Bus 0000:02
  fd400000-fd5fffff : PCI Bus 0000:01
    fd400000-fd403fff : 0000:01:01.0
      fd400000-fd403fff : virtio-pci-modern
  fd600000-fd603fff : 0000:00:03.0
    fd600000-fd603fff : virtio-pci-modern
  fd604000-fd607fff : 0000:00:12.0
    fd604000-fd607fff : virtio-pci-modern
  fe400000-fe5fffff : PCI Bus 0000:03
  fe600000-fe7fffff : PCI Bus 0000:02
  fe800000-fe9fffff : PCI Bus 0000:01
    fe800000-fe800fff : 0000:01:01.0
  fea00000-fea3ffff : 0000:00:12.0
  fea50000-fea50fff : 0000:00:02.0
    fea50000-fea50fff : bochs-drm
  fea51000-fea510ff : 0000:00:05.0
  fea52000-fea52fff : 0000:00:12.0
  fea53000-fea530ff : 0000:00:1e.0
```

How much RAM installed in the memory : free -m

```
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT$ free -m
               total        used        free      shared  buff/cache   avail
able
Mem:            3196         381        1376           9        1438
2633
Swap:           3429           0        3429
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT$
```

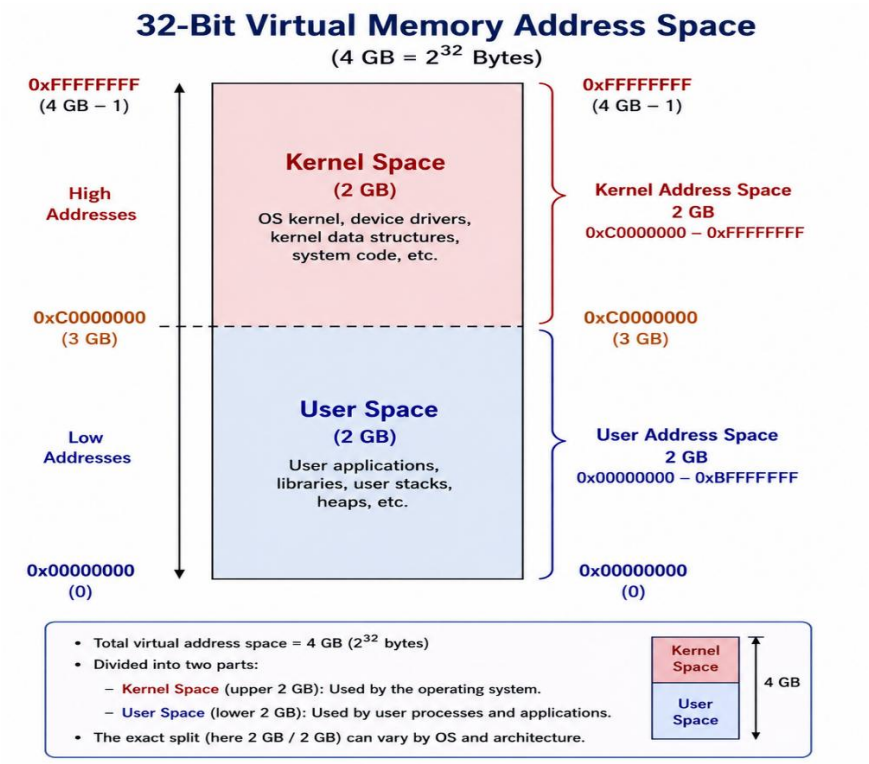
Cat /proc/meminfo

```
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT$ cat /proc/meminfo
MemTotal:        3273412 kB
MemFree:         1409380 kB
MemAvailable:    2696836 kB
Buffers:          106752 kB
Cached:          1287400 kB
SwapCached:            0 kB
Active:           666304 kB
Inactive:         938872 kB
Active(anon):     220964 kB
Inactive(anon):        0 kB
Active(file):     445340 kB
Inactive(file):   938872 kB
Unevictable:           0 kB
Mlocked:               0 kB
SwapTotal:       3512316 kB
SwapFree:        3512316 kB
Zswap:                 0 kB
Zswapped:              0 kB
Dirty:                 0 kB
Writeback:             0 kB
AnonPages:        211140 kB
Mapped:           199532 kB
Shmem:              9924 kB
KReclaimable:      79288 kB
Slab:             165976 kB
SReclaimable:      79288 kB
SUnreclaim:        86688 kB
KernelStack:        4336 kB
PageTables:         9128 kB
SecPageTables:         0 kB
```

Virtual Address Space for 32-bit Processors

- **Virtual Addresses:** On Linux, every memory address is virtual. They do not point directly to any address in the RAM.
- **Translation Mechanism:** Whenever you access a memory location, a translation mechanism is performed to match it to the corresponding physical memory.
- **Process Ownership:** On Linux systems, each process owns its own virtual address space.
- **Address Space Size:** The size of the virtual address space is **4 GB** on 32-bit systems (even on a system with physical memory less than 4 GB).
- **Space Division:** Linux divides this virtual address space into two areas:
    - An area for applications, called **user space**.

    - An area for the kernel, called **kernel space**.
- **The Split (PAGE_OFFSET):** The split between the two areas is set by a kernel configuration parameter named PAGE_OFFSET.
- **3G/1G Split:** This division is commonly referred to as the **3G/1G Split** (where 3 GB is typically allocated for user space and 1 GB for kernel space)



Process Separation and Kernel Memory Mapping

- **Each process has its own User Address Space.**
    - This memory area belongs only to that process.
- **User Address Space acts as a sandbox.**
    - One process cannot directly access another process's memory.
- **This isolation improves security and stability.**
    - Errors in one process do not affect other processes.
- **Kernel Address Space is shared by all processes.**
    - There is only one operating system kernel in the system.
- **The kernel memory is mapped into every process's virtual address space.**
    - This allows processes to access kernel services when needed.
- **Processes use System Calls to request services from the kernel.**
    - Examples: file operations, memory allocation, and device access.
- **Keeping the kernel mapped in every process improves performance.**
    - The CPU does not need to switch address spaces whenever a process enters or exits kernel mode.
- **This design provides both efficiency and protection.**
    - User processes remain isolated while kernel services remain quickly accessible.

Key Point

User Space = Private to each process
Kernel Space = Shared by all processes
Result = Better Security + Better Performance

Kinds of Memory

- **Both User Space and Kernel Space use Virtual Addresses.**
  - Programs and the kernel normally work with virtual addresses, not physical addresses.
- **Virtual Addresses are translated into Physical Addresses.**
  - This translation is performed by the **Memory Management Unit (MMU)**.
- **MMU (Memory Management Unit).**
  - A hardware component responsible for mapping virtual addresses to physical addresses.
- **Kernel Address Conversion Support.**
  - The Linux kernel provides functions to convert between virtual and physical addresses.
- **Required Header File:** #include <asm/io.h>

Address Conversion Functions

- **Convert Virtual Address to Physical Address**

phys_addr = virt_to_phys(virt_addr);

Returns the physical address corresponding to a kernel virtual address.

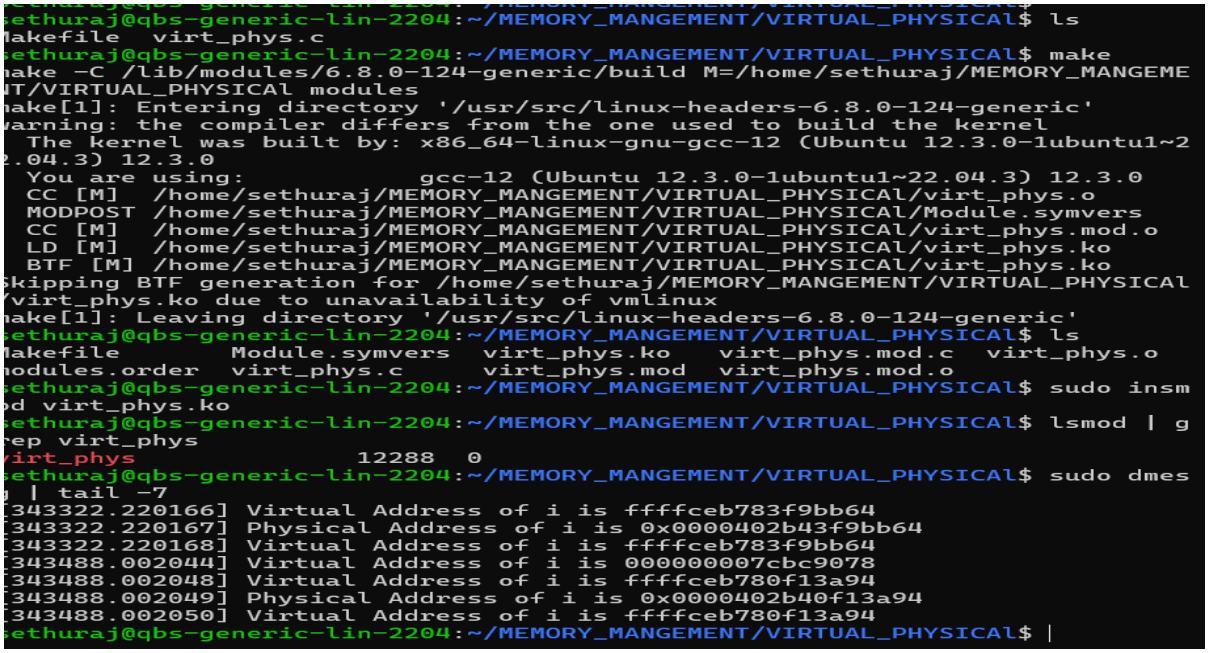
- **Convert Physical Address to Virtual Address**

virt_addr = phys_to_virt(phys_addr);

Returns the kernel virtual address corresponding to a physical address.

Key Point

- **Applications and the kernel use virtual addresses.**
- **The MMU translates virtual addresses to physical addresses.**
- **Kernel code can use virt_to_phys() and phys_to_virt() for address conversion**



Pages and Page Tables

Pages

- The virtual address space is divided into fixed-size blocks called **pages**.
- On most **x86** and **ARM** systems, the page size is **4096 bytes (4 KB)**.
- The page size is defined in the Linux kernel using the **PAGE_SIZE** macro.

PAGE_SIZE // Typically 4096 bytes

MMU and Page Mapping

- The **Memory Management Unit (MMU)** translates virtual addresses into physical addresses.
- The MMU uses **page tables** to perform this translation.
- Each virtual page is mapped to a physical page frame.

Virtual Address
  |
  v
Page Table
  |
  v
Physical Address

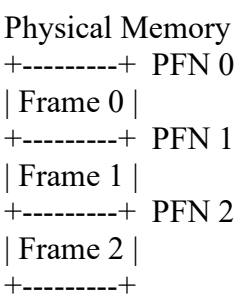
Memory Page (Virtual Page)

- A **memory page** is a fixed-size, contiguous block of **virtual memory**.
- Pages are the basic unit used by the operating system for memory management.

- In Linux, a page is represented by the **struct page** data structure.

Page Frame (Physical Memory)

- A **page frame** is a fixed-size, contiguous block of **physical memory (RAM)**.
- The operating system maps virtual pages onto physical page frames.
- Each page frame is identified by a unique **Page Frame Number (PFN)**.

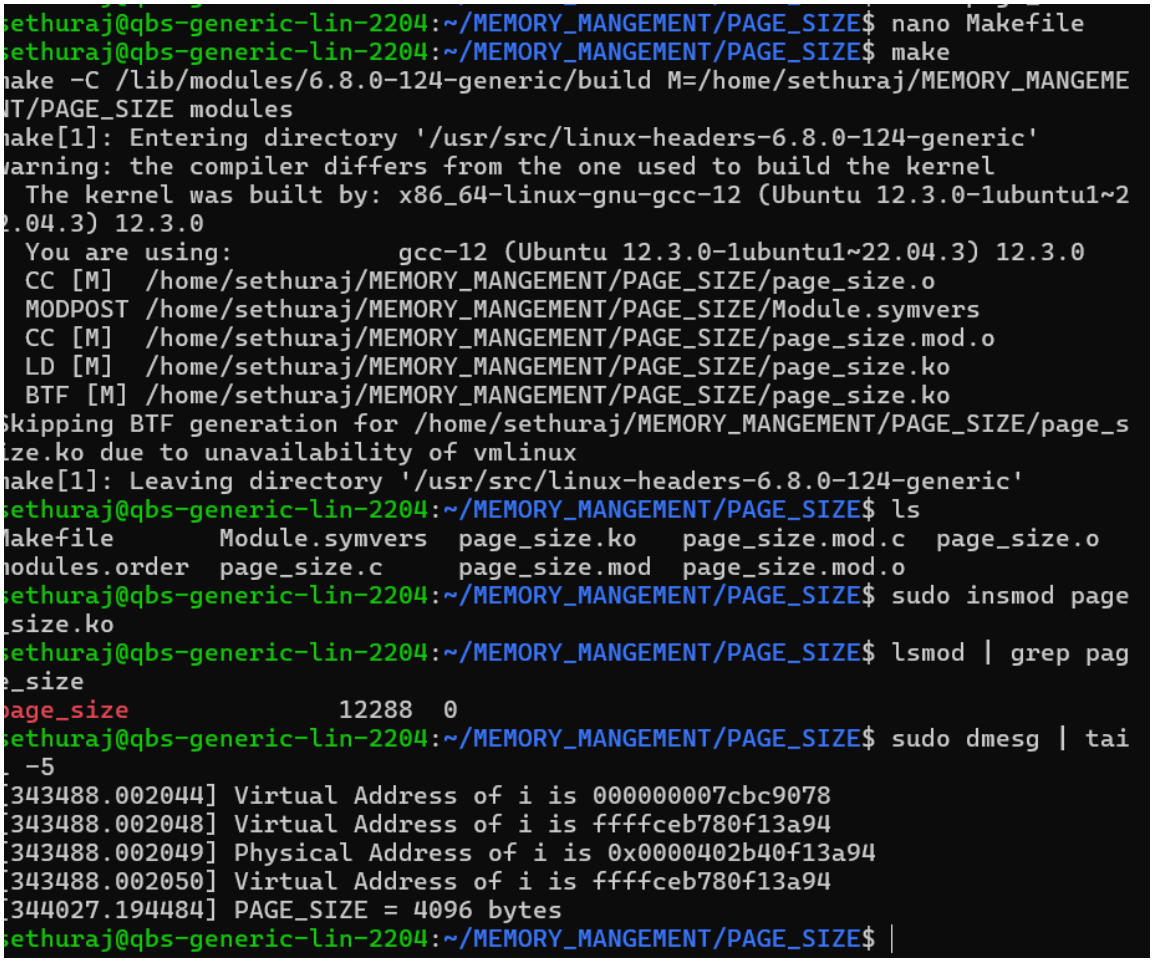


Page Table

- A **page table** is a kernel data structure that stores the mapping between virtual addresses and physical addresses.
- It is used by the MMU during address translation.
- Every process has its own page tables for its user-space memory.

Key Points

- **Page** = Fixed-size block of virtual memory (usually 4 KB).
- **Page Frame** = Fixed-size block of physical memory.
- **PFN** = Unique number identifying a page frame.
- **MMU** = Hardware that translates virtual addresses to physical addresses.
- **Page Table** = Data structure storing virtual-to-physical mappings.
- **struct page** = Kernel representation of a memory page.
- **page_to_pfn() / pfn_to_page()** = Convert between pages and page frame numbers



1. MMU and Page Tables

- The **MMU (Memory Management Unit)** is hardware inside the CPU.
- It converts **virtual addresses** into **physical addresses**.
- The MMU uses **page tables** to perform this translation.

Virtual Address
↓
Page Table
↓
Physical Address

2. Memory Page (Virtual Page)

- A **Page** is a fixed-size block of **virtual memory**.
- On most systems, one page is **4 KB (4096 bytes)**.
- Linux represents a page using **struct page**.

Virtual Memory
+---------+

| Page 0 |
+---------+
| Page 1 |
+---------+
| Page 2 |
+---------+

3. Page Frame (Physical Memory)

- A **Page Frame** is a fixed-size block of **physical RAM**.
- Virtual pages are mapped onto page frames.
- Each page frame has a unique **PFN (Page Frame Number)**.

Physical Memory
+----------+
| Frame 0 | ← PFN 0
+----------+
| Frame 1 | ← PFN 1
+----------+
| Frame 2 | ← PFN 2
+----------+

4. PFN Conversion

- Linux provides macros to convert between a page and its PFN.

```
page_to_pfn(page);   // Page → PFN
pfn_to_page(pfn);    // PFN → Page
```

5. Page Table

- A **Page Table** stores the mapping between virtual pages and physical page frames.
- Each entry contains information about a page-to-frame mapping.

Example:

Virtual Page	Physical Frame
Page 0	Frame 5
Page 1	Frame 12
Page 2	Frame 20

Term	Meaning
MMU	Hardware that translates virtual addresses to physical addresses
Page	Fixed-size block of virtual memory
Page Frame	Fixed-size block of physical memory
PFN	Number identifying a phys ical page frame
Page Table	Stores page-to-frame mappings

To get PAGESIZE : getconfig PAGE_SIZE or getconfig PAGESIZE

```
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_SIZE$ getconf PAGE_SIZE
4096
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_SIZE$ getconf PAGESIZE
4096
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_SIZE$
```

Expression	Meaning
PAGE_SIZE	Size of a memory page (usually 4096 bytes)
sizeof(struct page)	Size of the kernel's metadata structure used to represent a page

- **Page** = 4 KB block of virtual memory.
- **struct page** = Kernel structure that stores information about a page.
- **PAGE_SIZE** gives the size of a memory page.
- **sizeof(struct page)** gives the size of the metadata structure used by the kernel to manage pages

```
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/struct_page$ ls
Makefile  struct_page.c
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/struct_page$ make
make -C /lib/modules/6.8.0-124-generic/build M=/home/sethuraj/MEMORY_MANGEMENT/s
truct_page modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04
.3) 12.3.0
  You are using:           gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
  CC [M]  /home/sethuraj/MEMORY_MANGEMENT/struct_page/struct_page.o
  MODPOST /home/sethuraj/MEMORY_MANGEMENT/struct_page/Module.symvers
  CC [M]  /home/sethuraj/MEMORY_MANGEMENT/struct_page/struct_page.mod.o
  LD [M]  /home/sethuraj/MEMORY_MANGEMENT/struct_page/struct_page.ko
  BTF [M] /home/sethuraj/MEMORY_MANGEMENT/struct_page/struct_page.ko
Skipping BTF generation for /home/sethuraj/MEMORY_MANGEMENT/struct_page/struct_p
age.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/struct_page$ sudo insmod struct
_page.ko
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/struct_page$ lsmod | grep struc
t_page
struct_page            12288  0
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/struct_page$ sudo dmesg | tail
-5
[344027.194484] PAGE_SIZE = 4096 bytes
[344757.946253] PAGE_SIZE = 4096 bytes
[344757.946258] sizeof(struct page) = 64 bytes
[344799.472103] PAGE_SIZE = 4096 bytes
[344799.472107] sizeof(struct page) = 64 bytes
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/struct_page$
```

Kernel Memory Management vs User Space Allocation (Important Points)

- **Kernel memory allocation uses physical memory immediately.**
  - Functions like kmalloc() allocate real physical memory at once.
- **Kernel memory is never paged out.**
  - Once allocated, it remains in memory until freed.
- **User-space memory allocation is lazy.**
  - malloc() allocates virtual memory first, not physical memory.
- **Physical memory is allocated only when accessed.**
  - The kernel maps physical pages when the process actually reads or writes the memory.
- **Accessing an unmapped page causes a Page Fault.**
  - The CPU traps the access and notifies the kernel.
- **The kernel handles the Page Fault.**

- It allocates a physical page and updates the page table.

PAGE FAULT

What is a Page Fault?

A **Page Fault** occurs when a process tries to access a virtual memory page that is **not currently mapped to physical memory**.

When this happens:

1. The CPU detects that the page is not present.
2. The CPU raises a **page fault exception**.
3. The kernel handles the page fault.
4. The kernel allocates or loads the required page.
5. The page table is updated.
6. The process continues execution.

```
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/struct_page$ cd ../
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT$ mkdir PAGE_FAULT
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT$ cd PAGE_FAULT/
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_FAULT$ nano pagefault.c
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_FAULT$ nano Makefile
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_FAULT$ make
gcc -Wall -Wextra -o pagefault pagefault.c
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_FAULT$ ls
Makefile  pagefault  pagefault.c
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_FAULT$ ./pagefault
=== Initial State ===
Major Page Faults : 0
Minor Page Faults : 77

=== After malloc() ===
Major Page Faults : 0
Minor Page Faults : 80

=== After accessing memory ===
Major Page Faults : 0
Minor Page Faults : 335
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_FAULT$
```

 Before malloc() → Initial page fault count.

 After malloc() → Usually little or no increase because only virtual memory is allocated.

 After writing to memory → Minor page faults increase because Linux allocates physical pages on demand

Kernel Space	User Space
kmalloc() allocates physical memory immediately	malloc() allocates virtual memory initially
Memory is never paged out	Physical pages are allocated on demand
No lazy allocation	Uses lazy allocation

types of Page Faults

1. Minor Page Fault

- The page is already in memory.
- Only page table updates are needed.
- No disk access is required.
- Fast.

2. Major Page Fault

- The page is not in RAM.
- The kernel must load it from disk (swap or file).
- Slower because disk I/O is involved.

```
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT/PAGE_FAULT$ cat /proc/self/
maps
5d79daf4d000-5d79daf4f000 r--p 00000000 08:03 1048729                    /us
r/bin/cat
5d79daf4f000-5d79daf53000 r-xp 00002000 08:03 1048729                    /us
r/bin/cat
5d79daf53000-5d79daf55000 r--p 00006000 08:03 1048729                    /us
r/bin/cat
5d79daf55000-5d79daf56000 r--p 00007000 08:03 1048729                    /us
r/bin/cat
5d79daf56000-5d79daf57000 rw-p 00008000 08:03 1048729                    /us
r/bin/cat
5d79db353000-5d79db374000 rw-p 00000000 00:00 0                          [he
ap]
753fe8800000-753fe8d73000 r--p 00000000 08:03 1048860                    /us
r/lib/locale/locale-archive
753fe8e00000-753fe8e28000 r--p 00000000 08:03 1050728                    /us
r/lib/x86_64-linux-gnu/libc.so.6
```

Low Memory and High Memory

- The Linux kernel has its own **virtual address space**, separate from user-space processes.

- To access physical memory, the kernel must first **map it into its virtual address space**.
- A large portion of the kernel address space is used for **mapping physical memory**.
- The amount of physical memory the kernel can directly manage depends on the available kernel virtual address space.

3G/1G Memory Split (32-bit x86)

In a typical 32-bit Linux system:

User Space : 3 GB
Kernel Space : 1 GB

The **1 GB kernel space** is divided into:

Low Memory (LOWMEM)

- The first **896 MB** of kernel virtual address space.
- Permanently mapped to physical memory.
- Directly accessible by the kernel.

High Memory (HIGHMEM)

- The remaining **128 MB** of kernel virtual address space.
- Used to temporarily map physical memory beyond LOWMEM.
- Requires special mapping mechanisms before access.

Key Point

LOWMEM is directly mapped and easily accessible by the kernel, whereas HIGHMEM requires temporary mapping before it can be accessed.

Slab Allocation

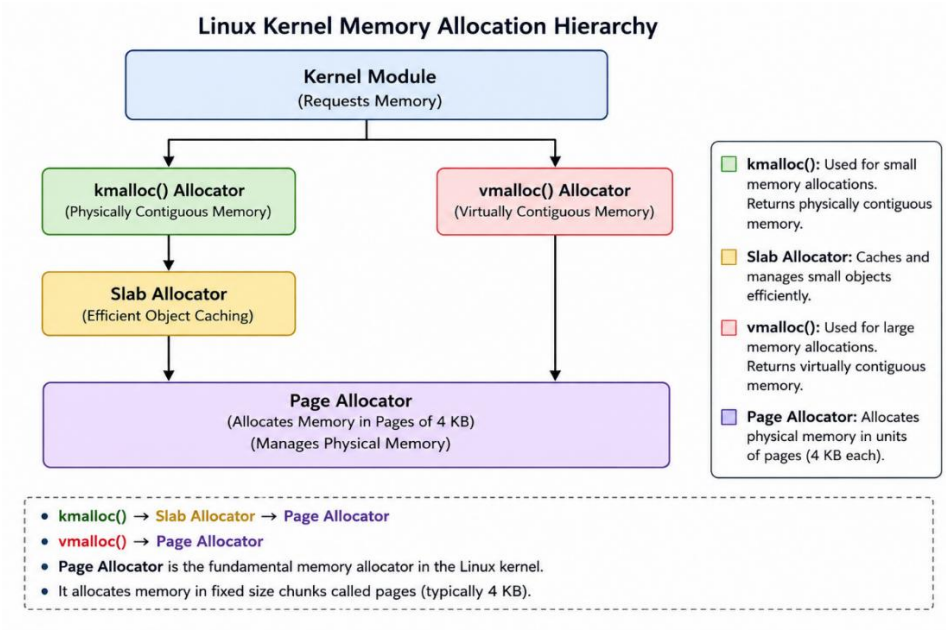
```
cat: /proc/slabinfo: Permission denied
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT$ sudo cat /proc/slabinfo
[sudo] password for sethuraj:
slabinfo - version: 2.1
# name            <active_objs> <num_objs> <objsize> <objperslab> <pagespers
lab> : tunables <limit> <batchcount> <sharedfactor> : slabdata <active_slabs
> <num_slabs> <sharedavail>
ext4_groupinfo_4k    660    660    184   22    1 : tunables    0    0    0 :
 slabdata     30     30      0
fsverity_info          0      0    272   15    1 : tunables    0    0    0 :
 slabdata      0      0      0
fscrypt_inode_info      0      0    136   30    1 : tunables    0    0    0
: slabdata      0      0      0
MPTCPv6                0      0   2112   15    8 : tunables    0    0    0 :
 slabdata      0      0      0
ip6-frags              0      0    184   22    1 : tunables    0    0    0 :
 slabdata      0      0      0
PINGv6                 0      0   1216   13    4 : tunables    0    0    0 :
 slabdata      0      0      0
RAWv6                 39     39   1216   13    4 : tunables    0    0    0 :
 slabdata      3      3      0
UDPv6                 36     36   1344   12    4 : tunables    0    0    0 :
 slabdata      3      3      0
tw_sock_TCPv6         15     15    264   15    1 : tunables    0    0    0 :
 slabdata      1      1      0
request_sock_TCPv6      0      0    312   13    1 : tunables    0    0    0
: slabdata      0      0      0
TCPv6                 36     36   2560   12    8 : tunables    0    0    0 :
 slabdata      3      3      0
kcopyd_job             0      0   3240   10    8 : tunables    0    0    0 :
 slabdata      0      0      0
dm_uevent              0      0   2888   11    8 : tunables    0    0    0 :
 slabdata      0      0      0
```

Linux Kernel Memory Allocation Hierarchy

The Linux kernel provides different memory allocation mechanisms to kernel modules. A kernel module can request memory using **kmalloc()** or **vmalloc()**. The **kmalloc()** allocator obtains physically contiguous memory and internally uses the **slab allocator** for efficient memory management. The **slab allocator** in turn requests memory pages from the **Page Allocator**. The **vmalloc()** allocator provides virtually contiguous memory and also relies on the **Page Allocator** for obtaining physical pages. At the lowest level, the **Page Allocator** is responsible for allocating and managing memory in units of pages, typically **4 KB** each.

Key Point

kmalloc() → Slab Allocator → Page Allocator
vmalloc() → Page Allocator
Page Allocator is the fundamental memory allocator in the Linux kernel.



kmalloc()

- Used to allocate memory in **kernel space**.
- Returns **physically contiguous** memory.
- Fast and efficient for small memory allocations.
- Memory can be directly accessed by hardware (DMA in some cases).
- Allocated memory must be freed using **kfree()**.

vmalloc()

- Used to allocate **large memory regions** in kernel space.
- Returns **virtually contiguous** memory.
- Physical pages may be scattered in RAM.
- Slightly slower than kmalloc() because page tables must be set up.
- Suitable for large allocations.

kfree()

- Used to free memory allocated by **kmalloc()**.
- Releases the memory back to the kernel.

- □ **kmalloc()** → Small, fast, physically contiguous memory.
- □ **vmalloc()** → Large, virtually contiguous memory.
- □ **kfree()** frees memory allocated by kmalloc().
- □ **vfree()** frees memory allocated by vmalloc()

Feature	kmalloc()	vmalloc()
Memory Type	Physically contiguous	Virtually contiguous
Speed	Faster	Slower
Best For	Small allocations	Large allocations
Backed By	Slab Allocator + Page Allocator	Page Allocator
Free Function	kfree()	vfree()

ZONES

Linux kernel divides physical RAM in to a number of different memory regions : zones

Linux divides RAM into zones (DMA, DMA32, and NORMAL) so that memory allocations can satisfy the requirements of different hardware devices and system architectures.

Zone	32-bit Linux	64-bit Linux	Purpose
DMA	0–16 MB	0–16 MB	Legacy DMA devices
DMA32	Not Present	16 MB–4 GB	32-bit DMA devices
NORMAL	16 MB–896 MB	Above 4 GB	General kernel allocations

1. ZONE_DMA

- Refers to the **first 16 MB of physical memory**.
- Reserved for old hardware devices that can perform DMA only within this range.
- Exists mainly for compatibility with legacy hardware.
  ZONE_DMA = 0 MB → 16 MB

2. ZONE_DMA32 (64-bit systems only)

- Exists only on **64-bit Linux**.
- Covers approximately the **first 4 GB of RAM**.
- Used by devices that can perform DMA only within the lower 4 GB address range.
  ZONE_DMA32 = 16 MB → 4 GB

3. ZONE_NORMAL
On 32-bit Systems

- Covers memory from **16 MB to 896 MB**.
- Kernel can directly access this memory.
  ZONE_NORMAL = 16 MB → 896 MB
  On 64-bit Systems
- Covers memory above **4 GB**.
- Most normal kernel allocations come from this zone.
  ZONE_NORMAL = Above 4 GB

Buddy Allocator

The **Buddy Allocator** is the fundamental memory allocation mechanism used by the Linux **Page Allocator** to manage physical memory.

Purpose

- Allocates and frees memory in units of **pages**.
- Manages contiguous blocks of physical memory efficiently.
- Reduces memory fragmentation.

How It Works

- Physical memory is divided into blocks whose sizes are powers of two.
- These blocks are called **buddies**.
- When a request arrives, the allocator finds the smallest suitable block.
- If the block is too large, it is repeatedly split into two equal buddies.
- When memory is freed, adjacent free buddies are merged back together

Suppose a process requests **4 pages**.

Initial Free Block

```
+----------------------+
|      16 Pages        |
+----------------------+
```

Split into two buddies:

```
+---------+---------+
| 8 Pages | 8 Pages |
+---------+---------+
```

Split again:

```
+----+----+---------+
| 4P | 4P | 8 Pages |
+----+----+---------+
```

One 4-page block is allocated.

Buddy Merge

When the allocated block is freed:

```
+----+----+
| 4P | 4P |
+----+----+
```

The two free buddies merge:

+---------+
| 8 Pages |
+---------+

If another adjacent 8-page buddy is free, they merge again:

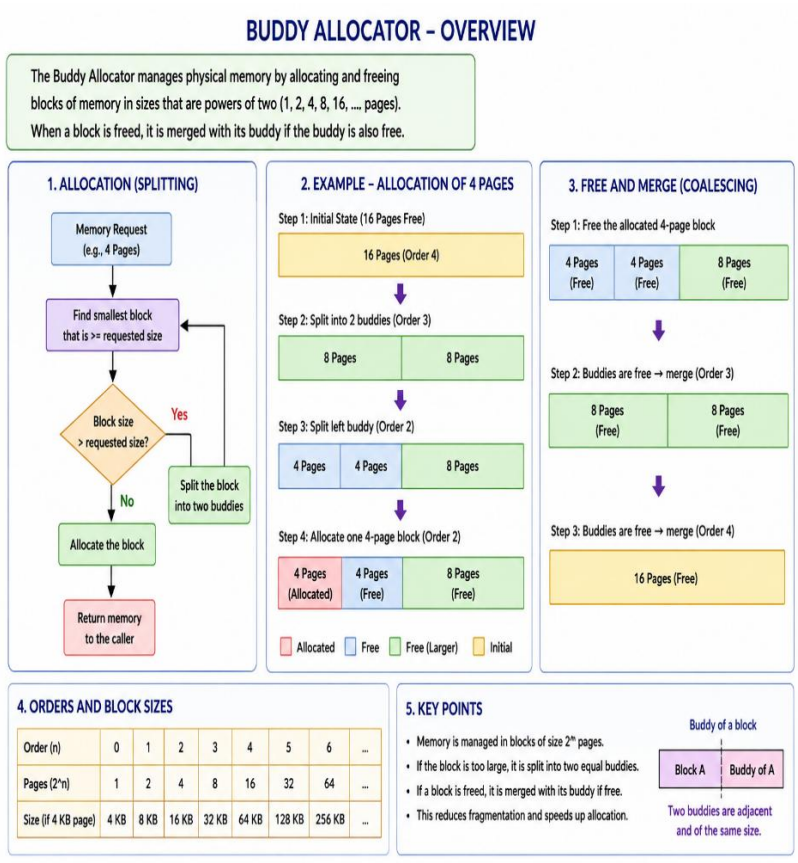
+-----------------+
| 16 Pages |
+-----------------+

Advantages

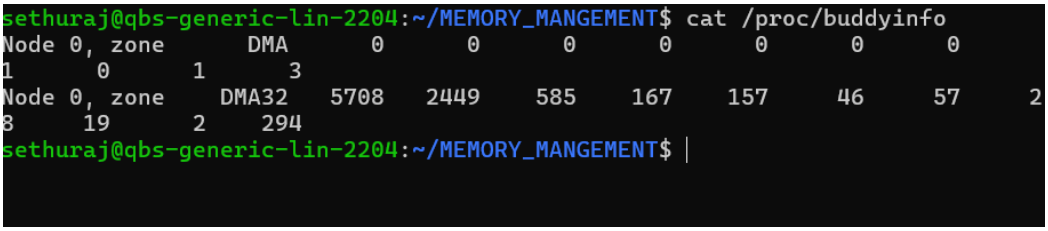
- Fast allocation and deallocation.
- Easy merging of free blocks.
- Efficient management of physical memory.

Disadvantage

- Can suffer from **internal fragmentation** because memory is allocated in power-of-two sizes.



/proc/buddyinfo shows the current state of the **Buddy Allocator** in the Linux kernel. It tells you how many **free memory blocks** are available in each **zone** and for each **order**.



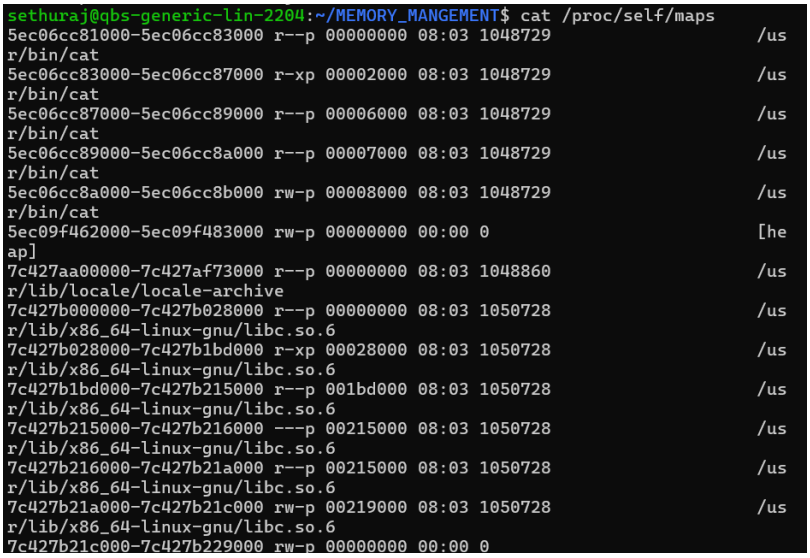
Node 0, zone DMA
0 0 0 0 0 0 0 1 0 1 3
This means:
Order 7 -> 1 free block (512 KB)
Order 9 -> 1 free block (2 MB)
Order 10 -> 3 free blocks (4 MB each)
All other orders currently have no free blocks

Each number represents the count of free blocks of a particular **order**:

Order	Pages	Size (4 KB page)
0	1 page	4 KB
1	2 pages	8 KB
2	4 pages	16 KB
3	8 pages	32 KB
4	16 pages	64 KB
5	32 pages	128 KB
6	64 pages	256 KB
7	128 pages	512 KB
8	256 pages	1 MB
9	512 pages	2 MB
10	1024 pages	4 MB

/proc/buddyinfo displays the number of free memory blocks available in each Buddy Allocator order for every memory zone (DMA, DMA32, NORMAL, etc.). It is used to monitor memory fragmentation and free memory availability.

Virtual kernel memory layout



32 bit s/m : sudo dmesg | grep -A 10

64 bit s/m : Kernel source code → Documentation folder → x86 → x86-64→vi mm.rst file

Example of passing various GFP_FLAGS TO kmalloc
GFP flags tell the kernel from which memory zone memory should be allocated and what allocation behavior is allowed (sleeping, atomic, DMA restrictions, etc.).

GFP_KERNEL

- Default flag used by most kernel code.
- Can sleep if memory is not immediately available.
- Suitable for normal kernel allocations.

GFP_DMA

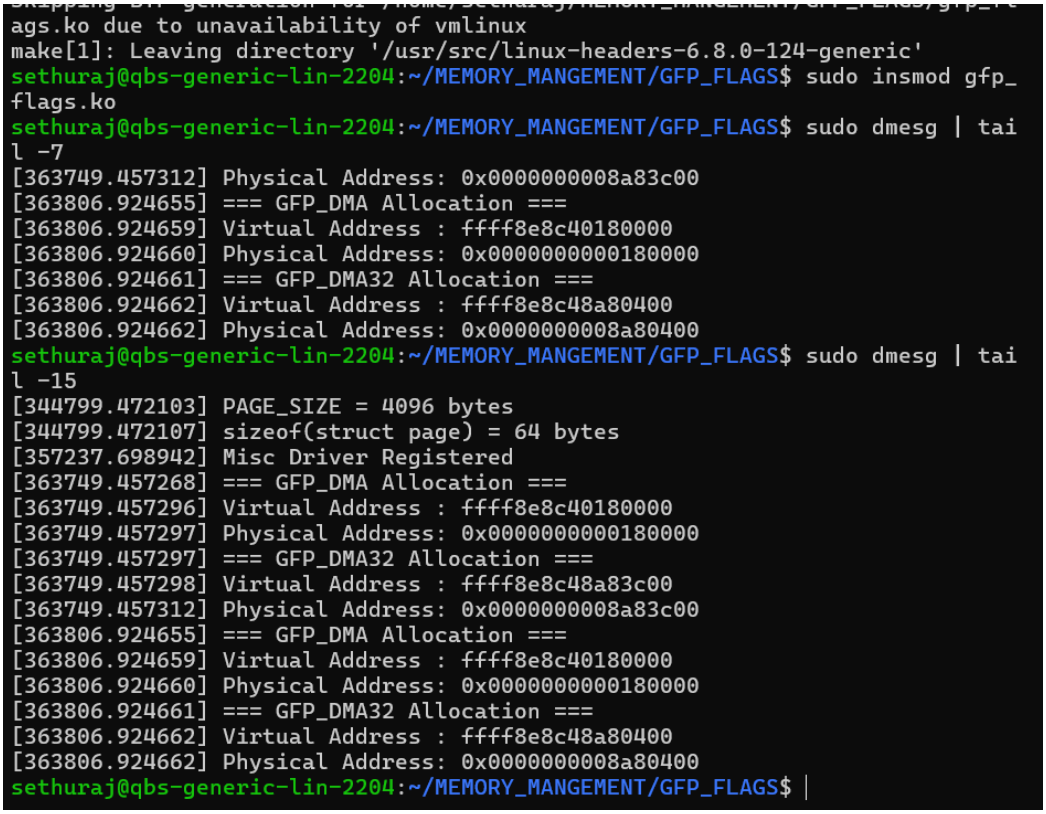
- Allocates memory from the **DMA zone**.
- Physical memory is usually below **16 MB**.
- Used by legacy DMA devices.

GFP_DMA32

- Allocates memory from the **DMA32 zone**.
- Physical memory is below **4 GB**.
- Used by devices that support only 32-bit DMA addressing.

GFP_ATOMIC

- Used in interrupt handlers and atomic contexts.
- Cannot sleep.
- Allocation may fail more easily.



GFP Flag	Purpose
GFP_KERNEL	Normal kernel memory allocation (can sleep)
GFP_ATOMIC	Allocation in interrupt/atomic context (cannot sleep)
GFP_DMA	Allocate memory from DMA zone (< 16 MB)
GFP_DMA32	Allocate memory from DMA32 zone (< 4 GB)
GFP_USER	User-space related memory allocation
GFP_NOWAIT	Do not sleep; return immediately if memory unavailable

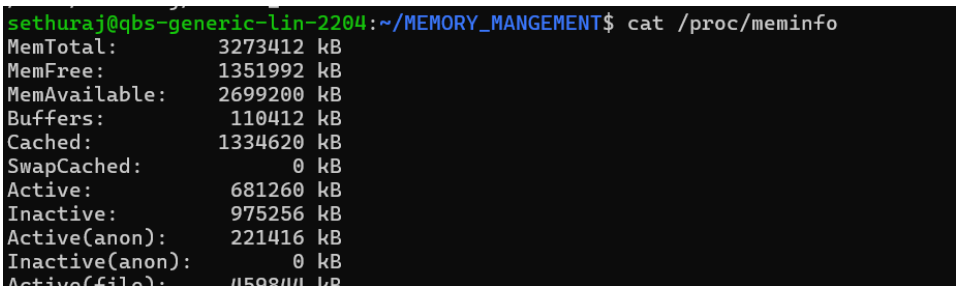
If memory allocated by **kmalloc()** is **not freed using kfree()**, it causes a **kernel memory leak**.

What Happens?

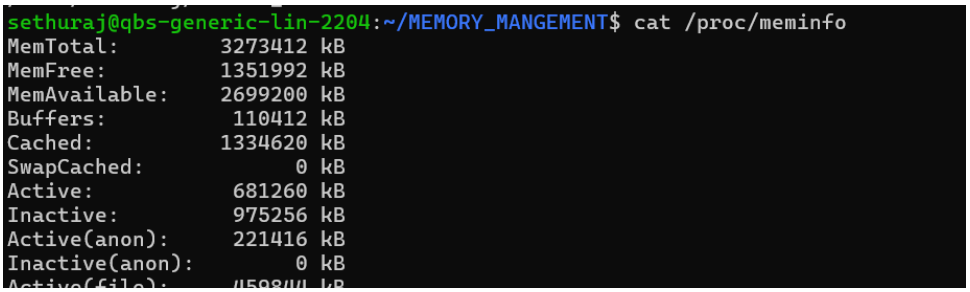
```
char *ptr;

ptr = kmalloc(1024, GFP_KERNEL);

/* Forgot to call kfree(ptr) */
```

Effects

- The allocated memory remains reserved.
- The kernel cannot reuse that memory.
- Available free memory gradually decreases.
- Repeated leaks can exhaust kernel memory.
- System performance may degrade.
- In severe cases, memory allocations start failing and the system can become unstable
  Memory allocated using kmalloc() must be released using kfree(). If it is not freed, a kernel memory leak occurs, reducing available kernel memory and potentially causing allocation failures or system instability.



cat /proc/meminfo | less displays detailed Linux memory information page by page, allowing you to examine RAM, cache, buffers, and swap usage interactively.



free Command

The free command displays the system's **RAM and swap memory usage**.

free -m

Displays memory information in **Megabytes (MB)**.

free -h

Displays memory information in a **human-readable format** (KB, MB, GB automatically).

```
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT$ free -h
               total        used        free      shared  buff/cache   avail
able
Mem:           3.1Gi       370Mi       1.3Gi       9.0Mi       1.5Gi       2
.6Gi
Swap:          3.3Gi          0B       3.3Gi
sethuraj@qbs-generic-lin-2204:~/MEMORY_MANGEMENT$ free -m
               total        used        free      shared  buff/cache   avail
able
Mem:            3196         370        1320           9        1505
2635
Swap:           3429           0        3429
```

Column	Meaning
total	Total installed RAM
used	Memory currently in use
free	Completely unused memory
shared	Memory shared between processes
buff/cache	Memory used for buffers and cache
available	Memory available for new applications
Command	Output Unit
free	KB (default)
free -m	MB
free -g	GB
free -h	Human-readable (KB/MB/GB)

ksize()
actual_size = ksize(ptr);

- kmalloc() may allocate more memory than requested.
- ksize() returns the actual allocated size.

kzalloc() in Linux Kernel
kzalloc() is similar to kmalloc(), but it **initializes the allocated memory to zero**.
Syntax
void *kzalloc(size_t size, gfp_t flags);

Linux Memory Management – Important Points

1. Address Space (32-bit System)

- Total address space = **4 GB (2^{32})**.

- Split into:
    - **User Space:** 3 GB (0x00000000 - 0xBFFFFFFF)
    - **Kernel Space:** 1 GB (0xC0000000 - 0xFFFFFFFF)
- Every process has its own **user-space virtual address space**.
- **Kernel space is shared** among all processes.

2. Virtual Memory and MMU

- Both kernel and user programs use **virtual addresses**.
- The **MMU (Memory Management Unit)** translates virtual addresses to physical addresses.
- **Page tables** store the virtual-to-physical mappings.

3. Pages and Frames

- Memory is divided into **Pages** (virtual memory blocks).
- On x86/ARM:
    - PAGE_SIZE = 4096 bytes (4 KB)
- Physical memory is divided into **Page Frames**.
- Each frame has a **PFN (Page Frame Number)**.
  page_to_pfn(page);
  pfn_to_page(pfn);

4. Low Memory and High Memory (32-bit)

- **LOWMEM:** First **896 MB**
    - Permanently mapped.
    - Directly accessible by the kernel.
- **HIGHMEM:** Remaining **128 MB**
    - Requires temporary mapping before access.

5. Kernel vs User Memory Allocation

Kernel Allocation
kmalloc(size, GFP_KERNEL);

- Physical memory allocated immediately.
- Not demand-paged.
- Never swapped out.

User Allocation
malloc(size);

- Virtual memory allocated first.
- Physical memory allocated only when accessed.

6. Page Fault

- Occurs when a process accesses a page not currently mapped.
- Kernel allocates a physical page and updates page tables.

Access Memory
↓
Page Fault
↓
Kernel Maps Physical Page
↓
Continue Execution

7. **Memory Zones**

ZONE_DMA

- First **16 MB**.
- For legacy DMA devices.

ZONE_DMA32

- First **4 GB** (64-bit only).
- For 32-bit DMA-capable devices.

ZONE_NORMAL

- General-purpose memory allocations.

8. **Buddy Allocator**

- Core physical memory allocator.
- Allocates memory in powers of two.
- Splits large blocks.
- Merges free buddy blocks.

16 Pages
↓
8 + 8
↓
4 + 4

9. **kmalloc vs vmalloc**

kmalloc()

- Physically contiguous memory.
- Faster.
- Used for small allocations.

ptr = kmalloc(size, GFP_KERNEL);

vmalloc()

- Virtually contiguous memory.
- Physical pages may be scattered.

- Suitable for large allocations.
  ptr = vmalloc(size);

10. Freeing Memory

```
kfree(ptr);   // for kmalloc
vfree(ptr);   // for vmalloc
```

- Forgetting to free memory causes a **kernel memory leak**.

11. GFP Flags

```
GFP_KERNEL
GFP_ATOMIC
GFP_DMA
GFP_DMA32
```

- **GFP_KERNEL:** Normal allocation.
- **GFP_ATOMIC:** Cannot sleep.
- **GFP_DMA:** Allocate from DMA zone.
- **GFP_DMA32:** Allocate below 4 GB.

12. ksize()

```
actual_size = ksize(ptr);
```

- kmalloc() may allocate more memory than requested.
- ksize() returns the actual allocated size.
- Extra allocated memory can be safely used.

- **MMU + Page Tables** translate virtual addresses to physical addresses.
- **Pages (4 KB)** are mapped to **Page Frames**.
- **Kernel memory** is allocated immediately using kmalloc().
- **User memory** uses lazy allocation and page faults.
- **Buddy Allocator** manages physical memory.
- **kmalloc()** → physically contiguous memory.
- **vmalloc()** → virtually contiguous memory.
- Always free memory using **kfree()** or **vfree()**.
- **GFP flags** control how and where memory is allocated